Monthly Price Forecasting – Case Study Report

Nov 3, 2026 ·

1. Data cleaning: missing values, outliers and anomalies

The dataset had no missing values, gaps or duplicates, but it contained a 30-month block of copied data (12% of the series) that was replaced before modelling. All cleaning is done on a working copy; the original 249 observations are kept untouched for auditing.

Structural validation. 249 monthly rows from Jan 2005 to Sep 2025, a continuous month-start index, 0 nulls, 0 duplicate dates, 0 missing months. Dates were parsed to datetime and the monthly frequency declared explicitly.

Anomaly detection (three independent screens).

Screen	Method	Finding	Treatment
Sudden movements	Month-over-month change above 20%	5 months flagged; largest +92% (Dec 2023, 8,400 to 16,163) and -53% (Jun 2021, 10,863 to 5,070)	Investigated, not altered: large but plausible market moves
Repeated values	Identical consecutive prices	9 pairs (e.g. Jul-Oct 2018 at 7,200)	Retained: consistent with flat reporting, no evidence of error
Copied blocks	Sliding 6-month window hashed against all earlier windows	Jun 2021-Dec 2022 (19 months) is an exact copy of Oct 2007-Apr 2009; Jan-Nov 2023 (11 months) is an exact copy of Oct 2009-Aug 2010	Treated as missing and re-estimated

Why the copied block had to be replaced. A 19-month and an 11-month sequence matching earlier history value-for-value cannot occur by chance. Left in place, the series drops 53% in one month and rises 92% in another purely because of the paste, which would dominate any model's variance estimate.

How it was replaced. The 30 values were set to NaN and filled by time-based interpolation on the log scale between the last genuine value before the block (May 2021, 10,863) and the first after it (Dec 2023, 16,163). Log interpolation assumes a constant proportional change per month, which is the appropriate assumption for a price series; linear interpolation was computed as a comparison and differed by only 1.4% on average. The interpolated values are labelled as estimates throughout and their effect on model choice is quantified in a sensitivity analysis (Section 3).

Outliers. A 12-month centred rolling median and rolling MAD were used to compute a robust z-score; a threshold of 3.5 flagged five observations (Apr and May 2006, Jul 2009, Jan and Feb 2024). Each was checked against its neighbours and none fell inside the interpolated period. They were retained: a statistical flag alone does not show a price is wrong, and removing real shocks would make the model under-estimate future volatility.

Stationarity. The cleaned level series is non-stationary (ADF p = 0.84). First differencing makes it stationary (ADF statistic -15.0, p < 0.001), so d = 1 was used for ARIMA-family models. This was re-confirmed on the training data alone to avoid any leakage from the test period.

2. The prediction model and why it was selected

The selected model is **Holt's Linear Trend exponential smoothing**, chosen because it had the lowest average error when forecasting 12 months ahead from four different historical starting points. It was not the best model on any single test window; it was the most consistent across windows, which is what matters for a 12-month deliverable.

How the model works. Holt's method keeps two state variables that are updated every month:

$$\ell_t = \alpha\, y_t + (1-\alpha)(\ell_{t-1} + b_{t-1})$$

$$b_t = \beta\,(\ell_t - \ell_{t-1}) + (1-\beta)\, b_{t-1}$$

$$\hat{y}_{t+h} = \ell_t + h\, b_t$$

The level (l) is a smoothed estimate of the current price; the trend (b) is a smoothed estimate of the monthly change. The forecast h months ahead is the current level plus h times the current trend. The smoothing weights alpha and beta are estimated by maximum likelihood on the training data, so the model decides for itself how much weight recent months get versus older history. Prediction intervals are produced by simulating 2,000 future paths from the fitted state-space model and taking the 10th/90th and 2.5th/97.5th percentiles.

Candidates considered. Ten models were trained on Jan 2005-Sep 2024 and compared under identical conditions:

Family	Models	Notes
Baselines	Naive, Seasonal Naive, Historical Mean	Benchmarks every real model must beat
ARIMA	ARIMA(0,1,0), AIC-selected from p,q in 0..3 with and without drift	Reduces to a random walk; identical to Naive
SARIMA	(0,1,0)(0,1,1)12, AIC-selected from 32 configurations	Seasonal MA term significant (coef -0.91)
Exponential smoothing	Holt Linear, Holt Damped, Holt-Winters additive, Holt-Winters multiplicative	Level, trend and optional seasonality
Log-scale	Log-ARIMA, AIC-selected, back-transformed	Models proportional changes

Why Holt Linear over the alternatives.

1. **Horizon-matched evidence.** Mean 12-step RMSE across four origins: Holt Linear 1,175, SARIMA 1,193, Holt Damped 1,198, Holt-Winters additive 1,202, Naive/ARIMA 1,221. Holt Linear wins, although the margin over SARIMA is only 1.5%, so the choice is indicative rather than decisive.
2. **The series has a trend but no stable seasonality.** Calendar-month effects are weak and inconsistent (average monthly change ranges from -2.7% in June to +4.3% in May, with high variance). Seasonal models added parameters without adding accuracy, and Seasonal Naive was the worst non-trivial model at the 12-month horizon.

3. **Robust to the data-cleaning decision.** In the sensitivity analysis, the Holt-Winters models ranked first and second on the holdout with interpolated data but fell to sixth and seventh when the copied block was kept raw. Holt Linear and Naive barely moved. A model whose rank depends on how 30 months were imputed is not a safe choice.
4. **Interpretable.** Two state variables and two parameters. A business user can read the forecast as 'current level plus a monthly drift of about 40'.
5. **Cheap to retrain.** Refits in milliseconds, so it can be re-estimated every month in production without infrastructure.

What was rejected and why. Machine-learning regressors (gradient boosting, LSTM) were not used: 249 observations with no exogenous features is too little data for them to beat statistical baselines, and they would need lag features that reintroduce the same random-walk behaviour. ARIMA with higher orders was tried and rejected by AIC. Models trained only on post-gap data (Dec 2023 onward, 10 points) were evaluated as a sensitivity check but were worse on the holdout.

3. Model performance and evaluation metrics

At the 12-month horizon the selected model's average error is about 6.7% of price (MAPE) and about 1,175 in price units (RMSE). All models were evaluated on data they had not seen, in three complementary ways, and every figure below comes from the executed notebook.

Metrics used. MAE (average absolute error, in price units, easy to explain), RMSE (penalises large misses, the primary selection metric because a large forecast miss is costlier to a business than several small ones) and MAPE (scale-free, for communicating accuracy as a percentage). Training-fit statistics (SSE, AIC, BIC) were used only to pick orders within a model family, never to compare families.

Evaluation 1: horizon-matched rolling origins (primary). Each model was fitted on all data up to an origin month and forecast the next 12 months; this was repeated at Sep 2022, Mar 2023, Sep 2023 and Sep 2024, and the metrics averaged. This matches the real task: a 12-month forecast made at one point in time.

Model	MAE	RMSE	MAPE (%)
Holt Linear (selected)	941	1,175	6.73
SARIMA	966	1,193	6.90

Model	MAE	RMSE	MAPE (%)
Holt Damped	965	1,198	6.85
Holt-Winters additive	970	1,202	6.84
Log-ARIMA	984	1,220	6.95
Naive / ARIMA(0,1,0)	985	1,221	6.96
Holt-Winters multiplicative	1,003	1,242	7.05
Seasonal Naive	1,670	1,819	11.56
Historical Mean	6,500	6,583	43.99

Evaluation 2: single 12-month holdout (Oct 2024-Sep 2025). Models trained on Jan 2005-Sep 2024. Holt-Winters multiplicative was best here (RMSE 1,510, MAPE 9.0%); Holt Linear ranked 8th of 10 (RMSE 1,587, MAPE 9.5%); Naive scored RMSE 1,566, MAPE 9.1%. The spread between 1st and 8th is 5% of RMSE, which one 12-month window cannot distinguish reliably. This is why a single holdout was not used for selection.

Evaluation 3: one-step-ahead walk-forward (Jan-Sep 2024). Expanding window, model re-estimated every month. Naive MAPE 5.1%, Holt Linear 5.2%, SARIMA 5.4%. Seasonal Naive scored 4.7% but every one of its nine inputs was an interpolated 2023 value, so this result measures the interpolation, not the model, and was disregarded.

Training vs. predictive performance. Holt Linear's in-sample one-step RMSE on the training data is about 514 (SSE 62.7 million over 237 observations). Its out-of-sample 12-step RMSE of 1,175 is 2.3 times larger, which is the expected growth of error with horizon for a trend model and shows no sign of over-fitting; a model that fit the training data much better than it forecast would be the warning sign.

Sensitivity to the cleaning decision. Holdout RMSE with the copied block (a) interpolated, (b) left raw, (c) excluded from training:

Model	Interpolated	Raw block kept	Post-gap only
Holt Linear	1,587	1,603	2,136
Naive / ARIMA	1,566	1,566	1,566
Holt Damped	1,569	1,580	2,058
Holt-Winters multiplicative	1,510	1,921	not fittable (10 points)

Model	Interpolated	Raw block kept	Post-gap only
SARIMA	1,573	2,206	not fittable

The trend and random-walk models are stable across treatments; the seasonal models are not.

Final 12-month forecast (origin Sep 2025, last observed 15,750).

Month	Forecast	80% interval	95% interval
Oct 2025	15,790	15,100-16,535	14,736-16,951
Dec 2025	15,871	14,630-17,146	13,964-17,780
Mar 2026	15,991	14,242-17,745	13,292-18,745
Jun 2026	16,112	13,972-18,349	12,829-19,471
Sep 2026	16,232	13,684-18,764	12,373-20,094

The point forecast drifts up by about 40 per month (+3.1% over the year). The 95% interval is roughly plus or minus 7% at one month and plus or minus 24% at twelve months, which is the honest statement of uncertainty for a series with this history of abrupt level shifts.

4. Actions a company could take in response to the forecast

The forecast says prices are most likely to stay near today's level with a mild upward drift, but with a real chance of a large move in either direction (95% band of roughly 12,400-20,100 by Sep 2026). The right response is therefore not a bet on the point forecast but a set of actions that lock in the likely case and protect against the tails.

If the company buys this commodity (input cost).

1. **Forward-cover a share of next year's volume now.** With the forecast above today's price, fixing 40-60% of expected demand through forward contracts or supplier agreements at or near 15,750 removes exposure to the upside. Leave the rest floating so the company still benefits if prices fall into the lower half of the band.
2. **Build inventory selectively.** Where storage cost per month is below the forecast monthly drift (about 40, or 0.25% of price), buying ahead is cheaper than waiting. Where it is not, do not stockpile.

3. **Re-price contracts with customers.** Index customer prices to a published benchmark with a monthly or quarterly reset, so cost increases pass through automatically instead of eroding margin.
4. **Set budget assumptions from the interval, not the point.** Plan the base case at the forecast, stress-test cash flow at the 95% upper bound, and define in advance what is cut if that scenario arrives.

If the company sells this commodity (revenue).

1. **Hold off on long fixed-price sales contracts** longer than a quarter; the forecast gives no reason to lock in today's price for a year when the expected direction is up.
2. **Sell forward only the volume needed to secure cash flow** and keep the remainder exposed to spot.
3. **Time capacity decisions to the band.** Expansion that is only profitable above 18,000 should wait; expansion profitable at 14,000 can proceed.

For either side: operational triggers. Convert the forecast into rules rather than a document.

Trigger	Action
Actual price moves outside the 80% band for 2 consecutive months	Re-forecast immediately and review hedge ratio
Actual price moves outside the 95% band in any month	Escalate to management; treat as a regime change (as in Dec 2023)
Forecast drift turns negative on the monthly refresh	Reduce forward cover and inventory to minimum
Forecast drift exceeds 1% per month	Increase forward cover toward the upper limit

What not to do. Do not treat the point forecast as a target price; a 12-month-ahead error of about 7% is normal for this series and a 20% miss is within the 95% band. Do not hedge 100%; the model cannot distinguish a real trend from a random walk with confidence, so some exposure in both directions is the rational position.

5. Determining whether the actions were effective

Effectiveness is measured against a counterfactual, not against the forecast: the question is whether the company is better off than it would have been without the actions, given the prices that actually occurred. Three layers of measurement are needed, decided and written down before the actions start.

Layer 1: financial outcome versus the do-nothing baseline.

Metric	Definition	Target
Realised cost (or revenue) per unit	Weighted average price actually paid or received over the 12 months	Below (above) the simple average of monthly spot prices
Hedge benefit	(Average spot price - realised price) x hedged volume	Positive after hedging costs
Cost variance vs. budget	Realised spend vs. the budget set from the forecast	Within the 80% band set in Section 4
Margin stability	Standard deviation of monthly gross margin vs. prior 12 months	Lower than the pre-action period

The do-nothing baseline is computable exactly: the volume the company actually used, priced at each month's spot price. This is the control group.

Layer 2: process and decision quality. Outcomes depend partly on luck; a good hedging decision can lose money in a year when prices fall. So also measure whether the rules were followed:

- Share of months in which the trigger rules in Section 4 were applied as written.
- Time from a trigger firing to the action being taken.
- Hedge ratio actually held vs. the ratio the policy called for.

If the outcome was poor but the process was followed, the policy should be revised; if the outcome was good but the process was ignored, the result should not be credited to the actions.

Layer 3: forecast quality, because the actions depend on it. Track every month, using the Section 9 monitoring pipeline:

- Realised 1-, 3-, 6- and 12-month-ahead MAPE and RMSE vs. the back-tested values (6.7% MAPE at 12 months).
- Interval coverage: the share of months whose actual price fell inside the 80% and 95% bands should be close to 80% and 95%. Coverage well below that means the intervals were too narrow and the hedge ratios set from them were too aggressive.
- Direction accuracy: whether the sign of the forecast change matched the sign of the actual change.

Design for a fair test.

1. **Pre-register the metrics and thresholds** in the month the actions start, so success cannot be redefined afterwards.
2. **Where possible, use a comparison group.** If the company has several plants, regions or product lines, apply the new policy to some and keep the old policy on others for one or two quarters. The difference in realised cost per unit, adjusted for volume, is the cleanest estimate of the effect.
3. **Where no comparison group exists, use scenario accounting.** Compute what the 12 months would have cost under (a) no action, (b) the policy as written, and (c) the policy as actually executed. Report all three.
4. **Review quarterly, judge annually.** Monthly results are too noisy to act on; a 12-month window matches the forecast horizon and smooths out single-month shocks.

Decision rule at the end of the year. Continue the policy if realised cost per unit beat the do-nothing baseline and interval coverage was within 10 points of nominal. Revise the policy if it beat the baseline only because of a price move the model did not predict. Stop and re-design if it lost money while the forecast was accurate, since that means the actions, not the model, were wrong.

6. Deploying the model to production with Django

The model is packaged as a versioned artifact, loaded once per Django process, and served through a small `forecast` app behind Django REST Framework, with monthly retraining as a scheduled job. Django owns data, users and audit history; it does not retrain inside a request.

Step 1: package the model, not the notebook. Move the cleaning and modelling code out of the notebook into an installable package:

```
price_forecast/
  __init__.py
  cleaning.py      # copied-block detection, log interpolation, MAD screen
  registry.py      # fit_forecast(), forecast_with_intervals() from the
notebook
  train.py         # fits Holt Linear on the full cleaned history, writes
model.pkl + metadata.json
  tests/           # unit tests: cleaning reproduces the 30 flagged months,
forecast shape, interval ordering
```

`train.py` saves the fitted statsmodels results object with `joblib`, plus a `metadata.json` holding the model name, parameters (alpha, beta), training window, data hash, back-test metrics and a semantic version (e.g. `holt-linear-2025.09.1`). Pin `statsmodels`, `pandas` and `numpy` versions in `requirements.txt`; a pickle is only valid for the versions it was written with.

Step 2: Django project layout.

```
config/settings/{base,prod}.py
apps/forecast/
  models.py        # PriceObservation, ModelVersion, ForecastRun,
ForecastPoint
  services.py      # load_model(), run_forecast(origin), persist results
  serializers.py   # DRF serializers
  views.py         # ForecastViewSet, ObservationViewSet, HealthView
  urls.py
  management/commands/retrain.py
  management/commands/ingest_prices.py
  tasks.py         # Celery tasks wrapping the two commands
```

Database tables: `PriceObservation` (date, price, source, is_interpolated), `ModelVersion` (version, path, metrics JSON, is_active), `ForecastRun` (model_version, origin, created_at) and `ForecastPoint` (run, month, forecast, lower80, upper80, lower95, upper95). Storing every run gives the audit trail needed for Section 9.

Step 3: load the model once per process. In `services.py`, load the active `ModelVersion` lazily into a module-level cache on first use (or in `AppConfig.ready()`), so a request never pays the deserialisation cost. Gunicorn with `--preload` shares the loaded model across workers via copy-on-write.

Step 4: endpoints (DRF).

Method and path	Purpose
`GET /api/v1/forecast/latest/`	The 12 months of the most recent run, with intervals and model version
`GET /api/v1/forecast/runs/{id}/`	A historical run, for back-testing and audit
`POST /api/v1/observations/`	Ingest a new monthly price (authenticated, idempotent on date)
`POST /api/v1/forecast/run/`	Trigger a forecast from the current data (admin only; queued to Celery)
`GET /health/` and `GET /ready/`	Liveness (process up) and readiness (DB reachable, model loaded)

Authentication with DRF token or JWT; throttling on the public read endpoints; schema generated with `drf-spectacular` so the frontend teams in Section 8 get an OpenAPI contract.

Step 5: scheduled retraining. A Celery beat task (or a Kubernetes CronJob calling `manage.py retrain`) runs on the 2nd of each month after the new observation is ingested: re-run cleaning on the full history, refit Holt Linear, compute the rolling-origin back-test, and write a new `ModelVersion` with `is_active=False`. A second step promotes it to active only if its back-test RMSE is within 10% of the current model's; otherwise it raises an alert and the old model stays live. This is the champion-challenger pattern.

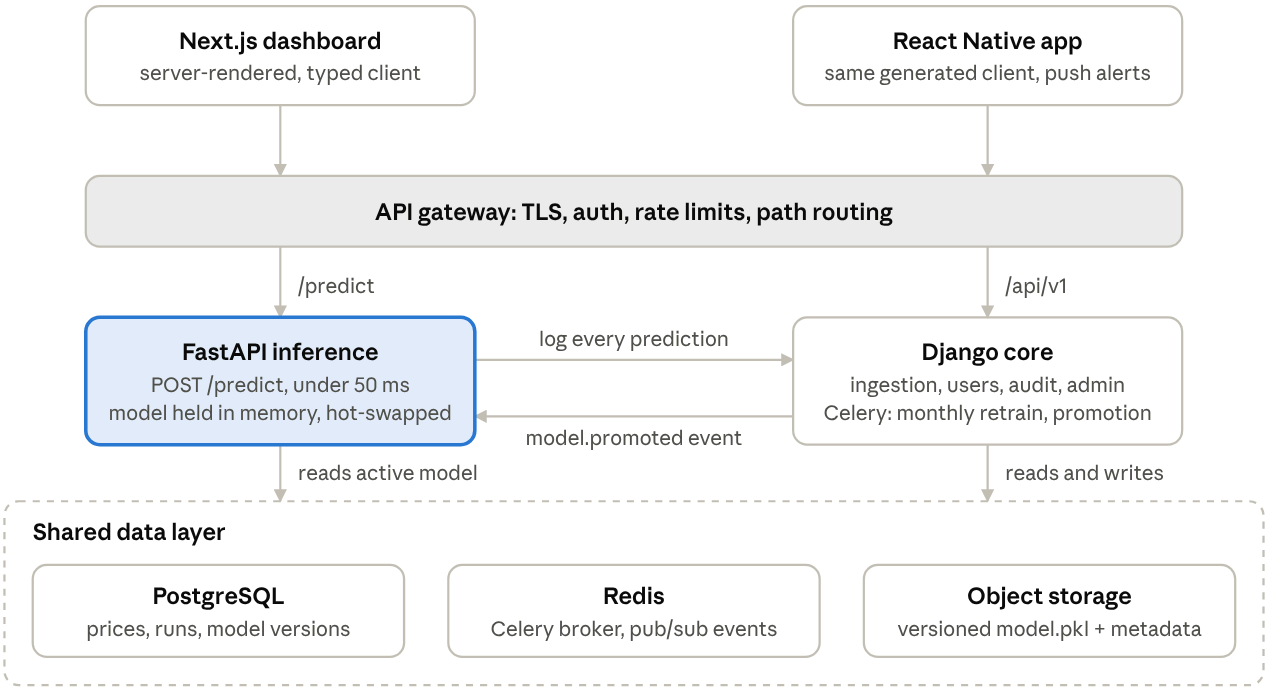
Step 6: infrastructure. Docker image built from a multi-stage Dockerfile (builder installs the pinned scientific stack, runtime stage copies site-packages). Gunicorn with 2-4 workers behind Nginx; PostgreSQL for the tables above; Redis as the Celery broker. Settings and secrets through environment variables (`django-environ`), never in the image. Deploy with a CI pipeline that runs the unit tests, builds the image, applies migrations (`manage.py migrate`) and performs a rolling restart; the readiness probe prevents traffic reaching a worker whose model has not loaded.

Step 7: safety nets. Structured JSON logging of every forecast run (version, origin, latency); Sentry for exceptions; database backups of `ForecastRun` so forecasts shown to the business can always be reproduced. The model file itself lives in object storage (S3 or equivalent) keyed by version, so a rollback is a one-row change to `ModelVersion.is_active`.

7. Django + FastAPI architecture for real-time predictions

Split the system in two: **FastAPI serves predictions**, **Django serves everything else**. FastAPI is async, lightweight and built around typed request/response schemas, which makes it the better inference server; Django has the ORM, admin, auth and background-job ecosystem, which makes it the better system of record. They share one PostgreSQL database and one model store, and talk over HTTP inside the private network.

FastAPI serves predictions, Django owns the data, both share one store



production architecture · 2 clients, 1 gateway, 2 services, 1 shared data layer

Requests enter through one gateway: prediction calls go left to FastAPI, everything else goes right to Django. The two services talk only by logging predictions and announcing a promoted model; both read the same stores underneath.

Responsibilities.

Concern	Owner	Why
Price ingestion, users, permissions, audit tables, admin UI	Django	ORM, migrations, admin and DRF already cover this

Concern	Owner	Why
Monthly retraining, champion-challenger promotion	Django + Celery	Long-running, scheduled, needs DB writes
`/predict` inference, interval computation, scenario what-ifs	FastAPI	Async, low latency, Pydantic validation, auto OpenAPI
Model artifact storage	Object storage (S3 or equivalent)	Both services read; only the trainer writes
Public API gateway and auth	Nginx or Traefik in front of both	One hostname, path-based routing, TLS termination

Request flow for a real-time prediction.

1. The client calls `POST /predict` on the gateway, which routes `/predict*` to FastAPI and `/api/*` to Django.
2. FastAPI validates the body with a Pydantic model: `origin` (optional month), `horizon` (1-24, default 12), `scenario` (optional list of assumed prices for months that have not been observed yet, so a user can ask 'what if October comes in at 17,000').
3. FastAPI holds the active Holt Linear model in memory, loaded at startup from the `ModelVersion` marked active. If a scenario is supplied, the model's state is updated through the scenario months before forecasting; statsmodels' `ExponentialSmoothing` refits in a few milliseconds on 250 points, so this can be done per request.
4. The response returns the 12 forecasts with 80% and 95% intervals, the model version and the data as-of date. Typical latency is under 50 ms.
5. FastAPI fires a background task that posts the request and response to Django's `/api/v1/predictions/log/`, so every prediction shown to a user is recorded for the monitoring in Section 9 without slowing the response.

Keeping the two services in step. Django publishes a `model.promoted` event (Redis pub/sub or a simple webhook) whenever the active version changes. FastAPI subscribes, downloads the new artifact, verifies its checksum against `metadata.json`, swaps it into memory atomically, and reports the new version on `GET /predict/version`. A rolling restart is not required. If FastAPI is scaled to several replicas, each one subscribes independently; the gateway's health check includes the version so a stale replica can be detected.

FastAPI service sketch.

```
inference/
  main.py          # FastAPI app, lifespan handler loads the model
  schemas.py       # PredictRequest, PredictResponse, ForecastPoint
(Pydantic)
  model_store.py   # fetch active version from Django API, download,
checksum, cache
  predictor.py     # wraps price_forecast.registry.forecast_with_intervals
  routers/predict.py, routers/health.py
```

The `price_forecast` package from Section 6 is a shared dependency of both services, installed from the same wheel, so the cleaning and forecasting code can never diverge between training and serving.

Real-time ingestion, not only real-time prediction. When a new monthly price arrives (Django `POST /api/v1/observations/`), Django validates it, runs the anomaly screens from Section 1 on the incoming value (is it a >20% jump, is it a repeat, does it fall outside the current 95% interval), stores it with any flags, and enqueues a lightweight re-forecast. FastAPI then serves forecasts from the updated state within seconds of the observation landing, while the full monthly retrain still runs on schedule.

Operational details. Both services run as separate containers in the same compose file or Kubernetes namespace. FastAPI runs under Uvicorn with 2 workers per replica; Django under Gunicorn. Service-to-service calls use an internal token, not user credentials. Rate limiting and CORS are configured at the gateway so the frontend teams in Section 8 see one consistent policy. Both services expose Prometheus metrics (`/metrics`) and emit traces with a shared request id so a prediction can be followed from the client through FastAPI into the Django log table.

8. Integrating the APIs with Next.js and React Native

Both frontends consume the same OpenAPI contract through a generated, typed client, so the web dashboard and the mobile app cannot drift from the backend or from each other. The contract is the single source of truth; the UI code is generated from it.

Step 1: one contract, one generated client. Django (`drf-spectacular`) and FastAPI (built in) both emit OpenAPI 3 schemas. Merge them in CI into one `openapi.json` and run `openapi-typescript` (types) plus `orval` or `openapi-fetch` (a typed fetch client with TanStack Query hooks). Publish the result as a private npm package `@company/forecast-client`, versioned with the backend release. A backend change that breaks the schema fails the frontend build instead of failing in production.

Step 2: Next.js (web dashboard).

Need	Approach
Fast first load of the latest forecast	Server Component fetches `GET /api/v1/forecast/latest/` on the server with `fetch(..., { next: { revalidate: 3600 } })`; the page is cached for an hour and re-rendered on demand when Django calls a revalidation webhook after a new run
Interactive what-if scenarios	Client Component calls FastAPI `POST /predict` through the generated hook (`usePredict`), with TanStack Query handling loading, retry and caching keyed on the scenario inputs
Charts	Recharts or Visx: actual history as a line, forecast as a dashed line, the 80% and 95% bands as shaded areas, the Dec 2023 regime change annotated
Auth	NextAuth (Auth.js) issues the session; a Route Handler at `/api/proxy/*` attaches the backend JWT server-side so tokens never reach the browser
Secrets and base URLs	`NEXT_PUBLIC_API_BASE` for the gateway hostname only; all credentials in server-side env
Errors	Route-level `error.tsx` boundaries; a stale-but-available fallback shows the last cached forecast with a banner if FastAPI is down

Step 3: React Native (mobile app, Expo).

Need	Approach
Same client, no duplication	The same `@company/forecast-client` package; `fetch` is native in React Native, so no adapter is needed

Need	Approach
Auth	JWT stored in `expo-secure-store` (Keychain / Keystore), attached by an interceptor; refresh handled centrally
Offline	TanStack Query with `persistQueryClient` to AsyncStorage: the last forecast is readable with no network; mutations (e.g. a manager approving a hedge action) are queued and replayed
Alerts	Expo push notifications triggered by Django when an actual price lands outside the 95% band (the Section 4 trigger), deep-linking to the forecast screen
Charts	`victory-native` or `react-native-skia` for the same line-plus-band chart as the web
Environments	`app.config.ts` reads `API_BASE` per build profile (dev, staging, prod) through EAS

Step 4: shared concerns across both.

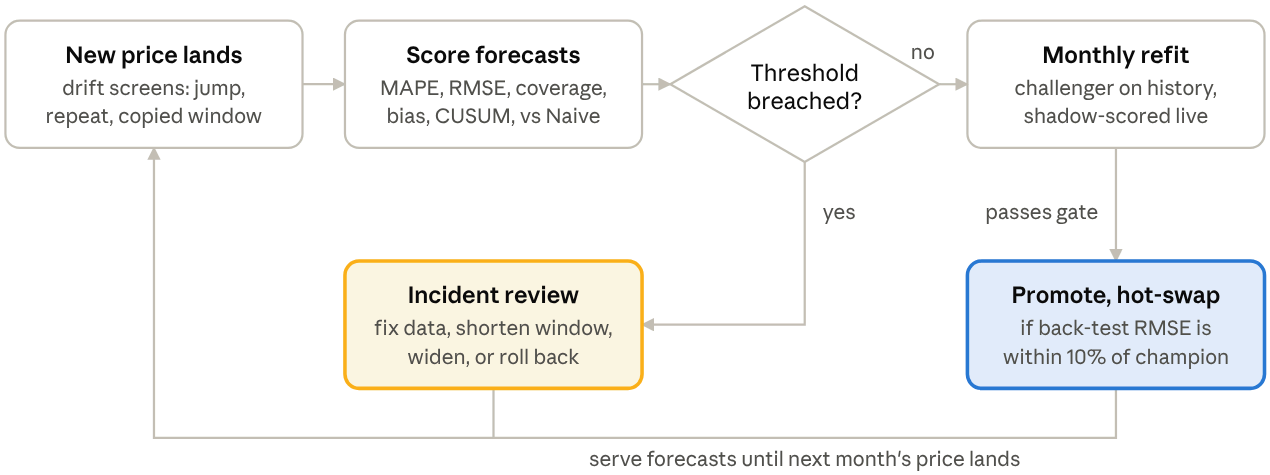
1. **Versioned API path** (`/api/v1/`) and a `model_version` field in every response, displayed in the UI footer, so a user and a support engineer can tell which model produced a number.
2. **Idempotency keys** on any POST that triggers a forecast run, so a retried tap on mobile does not queue two runs.
3. **Consistent error envelope** (`{code, message, details}`) from both backends, mapped to one toast component in each frontend.
4. **CORS and rate limits** set once at the gateway: allow the Next.js origin; mobile uses the bearer token, not cookies.
5. **Contract tests.** A Pact or schema-diff step in CI compares the frontend's recorded expectations with the live OpenAPI document on every backend PR.

Step 5: data flow in one sentence. Django ingests a price and persists the run, FastAPI serves the forecast in tens of milliseconds, Next.js renders it server-side for the dashboard and React Native shows the same numbers on a phone with a push alert when the band is breached, all through one generated client against one schema.

9. Monitoring in production: performance, drift and model updates

A monthly series produces one new truth value per month, so monitoring is a small number of well-chosen checks run on a schedule, with thresholds set from the back-test, not a real-time anomaly stream. Every check below runs automatically in the Django Celery pipeline on the day a new observation lands and writes its result to a `ModelHealth` table that feeds a Grafana dashboard and alert rules.

Every month the model is scored; a breach opens an incident, otherwise the challenger is refit



monthly monitoring loop · 4 steps, 1 decision, 1 incident path

The loop runs once per month when the new price arrives. The normal path is the top row and the right column; the incident box is entered only when a threshold in 9.1 or 9.2 fires, and a human decides what changes.

9.1 Tracking forecast performance. Each stored `ForecastRun` is scored as its target months become actual:

Metric	Computed from	Alert threshold (from back-test)
Rolling 12-month MAPE and RMSE at horizons 1, 3, 6, 12	Stored forecasts vs. ingested actuals	12-step MAPE above 10% for 3 consecutive months (back-test 6.7%)
Interval coverage	Share of actuals inside the 80% and 95% bands over the last 24 scored months	Coverage below 65% (80% band) or 85% (95% band)

Metric	Computed from	Alert threshold (from back-test)
Bias	Mean signed error at each horizon	Mean error outside plus or minus 1 RMSE for 3 months: the model is systematically high or low
Direction accuracy	Sign of forecast change vs. sign of actual change	Below 50% over 12 months
Champion vs. Naive	RMSE of the live model vs. a Naive forecast made at the same origin	Live model worse than Naive for 3 consecutive origins

The last row is the most important one: a forecasting model that cannot beat 'price stays where it is' should not be in production.

9.2 Detecting drift. Two kinds of drift matter here and they are detected differently.

Data drift (the input series changes character). On each new observation: (a) the Section 1 anomaly screens (>20% jump, repeat, copied-window check against history); (b) a rolling comparison of the last 24 months' mean, variance and first-difference variance against the training window using a Kolmogorov-Smirnov or Page-Hinkley test; (c) a check that the new value lies inside the previous run's 95% interval. Dec 2023 (+92% in one month) is the template for what this must catch: it would have failed (a) and (c) immediately.

Concept drift (the relationship the model learned stops holding). A CUSUM chart on the one-step forecast errors, reset each time the model is retrained. A sustained run of same-signed errors, or a CUSUM crossing of about 4 sigma, signals that the trend the model is extrapolating has changed. Also track the fitted alpha and beta each month: a sharp change in the smoothing parameters means the model is re-weighting history, which is itself a drift signal.

9.3 Updating the model. Three update paths, from routine to exceptional:

1. **Monthly refit (routine).** Re-estimate Holt Linear on the full cleaned history. Promote only if its rolling-origin back-test RMSE is within 10% of the live model's (champion-challenger, Section 6). Expected outcome most months: a small parameter change, no alert.

2. **Quarterly re-selection.** Re-run the full ten-model comparison from the notebook on the latest data. If a different family wins by more than 5% on mean 12-step RMSE for two consecutive quarters, switch families. This catches the case where, for example, seasonality becomes real as more post-2023 data accumulates.
3. **Triggered redesign (exceptional).** Any 95%-band breach, CUSUM alarm, or the live model losing to Naive opens an incident. The response is human, not automatic: inspect the new data for a data error first (the copied-block episode shows that the data can be wrong before the market is), then decide whether to shorten the training window to the new regime, add an intervention dummy, or hold the model and widen the intervals while the regime settles.

9.4 Shadow mode and rollback. A challenger model always runs in shadow alongside the champion, scored on the same actuals, so a switch is backed by live evidence and not only back-tests. Every `ModelVersion` artifact is retained; rollback is flipping `is_active` and letting FastAPI hot-swap (Section 7), and takes under a minute.

9.5 Operational health. Separately from model quality: prediction latency p95 (target under 100 ms), error rate of `/predict`, time since last successful ingestion (alert if a month's price is more than 5 days late), model version served by each FastAPI replica (alert on mismatch), and training-job success. All exported to Prometheus with Alertmanager routing to the data-team channel.

9.6 Governance. A monthly one-page model report, generated automatically from the `ModelHealth` table: current version, last 12 months of actual vs. forecast with bands, coverage, bias, drift flags raised, and any promotion or rollback. The same report is what Section 5 uses to judge whether business actions were based on a forecast that was performing as expected.